python.data-structure#

https://pytorch.org/docs/stable/generated/exportdb/python.data-structure.html

Description:

Tags: python.data-structure Support Level: SUPPORTED Original source code:

Code Examples

```
# mypy: allow-untyped-defs
import torch

class Dictionary(torch.nn.Module):
    """
    Dictionary structures are inlined and flattened along
tracing.
    """

    def forward(self, x, y):
        elements = {}
        elements["x2"] = x * x
        y = y * elements["x2"]
        return {"y": y}

example_args = (torch.randn(3, 2), torch.tensor(4))
tags = {"python.data-structure"}
model = Dictionary()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3, 2]", y: "i64[]"):
                mul: "f32[3, 2]" = torch.ops.aten.mul.Tensor(x,
x);  x = None

                mul_1: "f32[3, 2]" =
torch.ops.aten.mul.Tensor(y, mul);  y = mul = None
            return (mul_1,)

Graph signature:
    # inputs
    x: USER_INPUT
    y: USER_INPUT

    # outputs
    mul_1: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

class FnWithKwargs(torch.nn.Module):
    """
    Keyword arguments are not supported at the moment.
    """

    def forward(self, pos0, tuple0, *myargs, mykw0, **mykwargs):
        out = pos0
        for arg in tuple0:
            out = out * arg
        for arg in myargs:
            out = out * arg
        out = out * mykw0
        out = out * mykwargs["input0"] * mykwargs["input1"]
        return out

example_args = (
    torch.randn(4),
    (torch.randn(4), torch.randn(4)),
    *[torch.randn(4), torch.randn(4)]
)
example_kwargs = {
    "mykw0": torch.randn(4),
    "input0": torch.randn(4),
    "input1": torch.randn(4),
}
tags = {"python.data-structure"}
model = FnWithKwargs()


torch.export.export(model, example_args, example_kwargs)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, pos0: "f32[4]", tuple0_0: "f32[4]",
tuple0_1: "f32[4]", myargs_0: "f32[4]", myargs_1: "f32[4]",
mykw0: "f32[4]", input0: "f32[4]", input1: "f32[4]"):
                mul: "f32[4]" = torch.ops.aten.mul.Tensor(pos0,
tuple0_0);  pos0 = tuple0_0 = None
            mul_1: "f32[4]" = torch.ops.aten.mul.Tensor(mul,
tuple0_1);  mul = tuple0_1 = None

                mul_2: "f32[4]" =
torch.ops.aten.mul.Tensor(mul_1, myargs_0);  mul_1 = myargs_0 =
None
            mul_3: "f32[4]" = torch.ops.aten.mul.Tensor(mul_2,
myargs_1);  mul_2 = myargs_1 = None

                mul_4: "f32[4]" =
torch.ops.aten.mul.Tensor(mul_3, mykw0);  mul_3 = mykw0 = None

                mul_5: "f32[4]" =
torch.ops.aten.mul.Tensor(mul_4, input0);  mul_4 = input0 = None
            mul_6: "f32[4]" = torch.ops.aten.mul.Tensor(mul_5,
input1);  mul_5 = input1 = None
            return (mul_6,)

Graph signature:
    # inputs
    pos0: USER_INPUT
    tuple0_0: USER_INPUT
    tuple0_1: USER_INPUT
    myargs_0: USER_INPUT
    myargs_1: USER_INPUT
    mykw0: USER_INPUT
    input0: USER_INPUT
    input1: USER_INPUT

    # outputs
```

```
    mul_6: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

class ListContains(torch.nn.Module):
    """
    List containment relation can be checked on a dynamic shape
or constants.
    """

    def forward(self, x):
        assert x.size(-1) in [6, 2]
        assert x.size(0) not in [4, 5, 6]
        assert "monkey" not in ["cow", "pig"]
        return x + x

example_args = (torch.randn(3, 2),)
tags = {"torch.dynamic-shape", "python.data-structure",
"python.assert"}
model = ListContains()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3, 2]"):
                add: "f32[3, 2]" = torch.ops.aten.add.Tensor(x,
x);  x = None
            return (add,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    add: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs

import torch

class ListUnpack(torch.nn.Module):
    """
    Lists are treated as static construct, therefore unpacking
should be
    erased after tracing.
    """

    def forward(self, args: list[torch.Tensor]):
        """
        Lists are treated as static construct, therefore
unpacking should be
        erased after tracing.
        """
        x, *y = args
        return x + y[0]

example_args = ([torch.randn(3, 2), torch.tensor(4),
torch.tensor(5)],)
tags = {"python.control-flow", "python.data-structure"}
model = ListUnpack()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, args_0: "f32[3, 2]", args_1: "i64[]",
args_2: "i64[]"):
                add: "f32[3, 2]" =
torch.ops.aten.add.Tensor(args_0, args_1);  args_0 = args_1 =
None
            return (add,)

Graph signature:
    # inputs
    args_0: USER_INPUT
    args_1: USER_INPUT
    args_2: USER_INPUT

    # outputs
    add: USER_OUTPUT

Range constraints: {}
```

Notes & Warnings

NOTE Note Tags: python.data-structure Support Level: SUPPORTED

NOTE Note Tags: python.data-structure Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.data-structure, python.assert Support Level: SUPPORTED

NOTE Note Tags: python.data-structure, python.control-flow Support Level: SUPPORTED

Detailed Documentation

dictionary#

Original source code:

---

fn_with_kwargs#

Original source code:

list_contains#

Original source code:

list_unpack#

Original source code:

Tags: python.data-structure

Support Level: SUPPORTED

Original source code:

Tags: python.data-structure

Support Level: SUPPORTED

Original source code:

Tags: torch.dynamic-shape, python.data-structure, python.assert

Support Level: SUPPORTED

Original source code:

Tags: python.data-structure, python.control-flow

Support Level: SUPPORTED

Original source code: